

Biff NATS Relay: A Z Specification

Formal Model of Connection Lifecycle and KV Watcher

March 2026

Contents

1	Introduction	2
2	Basic Types	2
3	Free Types	2
4	Global Constants	3
5	Connection State	3
6	Connection Operations	4
6.1	Ensure Connected (Lazy Connect)	4
6.2	Disconnect (Reversible)	4
6.3	Close (Permanent)	4
6.4	Reset Infrastructure	5
6.5	Request (JetStream / KV Round-Trip)	5
6.6	Wedge Detected (Recovery)	5
6.7	Reconnect (nats-py Auto-Reconnect Completes)	6
6.8	Force Reconnect (Proactive Wedge Teardown)	6
7	Session Presence	6
7.1	Update Session (Heartbeat / Login)	7
7.2	Delete Session (Logout / TTL Expiry)	7
7.3	Set Wall	7
7.4	Clear Wall	7
8	KV Watcher State Machine	8
8.1	Start Watching	8
8.2	Snapshot Done	8
8.3	Receive Entry	9
8.4	Handle Timeout	9
8.5	Handle Error (Supervisor Restart)	9
8.6	Request Shutdown	9
9	Composed System State	10
10	Key Properties and Model Checking	10

1 Introduction

The NATS relay is the networked backend for *biff*'s communication system. It manages a TCP connection to a NATS server, lazily provisions infrastructure (KV bucket, JetStream streams), and runs a KV watcher that detects session changes and wall broadcasts in real-time.

This specification models three interacting state machines:

1. **Connection lifecycle** — lazy connect, disconnect, reconnect, and close transitions.
2. **KV watcher** — the long-lived watcher that observes session and wall changes, survives snapshot-done markers and timeouts, and restarts only on errors.
3. **Session presence** — KV-backed sessions with TTL expiry, heartbeat refresh, and logout detection.

The key invariant proven by this model: the KV watcher remains alive across snapshot-done markers and idle timeouts. Only explicit shutdown or unrecoverable errors terminate it. This captures the fix for *biff-udp*.

A second invariant, added to close a recurring class of wedge bugs, concerns the connection itself. A NATS connection can enter a *half-open* state: the TCP socket is up (`is_closed` is false) but every JetStream or KV request times out. If no transition can leave that state, the relay is wedged — every tool call blocks on `nats: timeout` with no recovery (*biff-tww*). This specification adds the half-open state explicitly and proves a liveness property: the connection can never be stuck there. Two independent transitions escape the wedge. The keepalive detector (*WedgeDetected*) waits for *nats-py*'s ping loop to declare the socket dead and hands it to the background reconnect; the proactive detector (*ForceReconnect*, *biff-3hp*) force-closes the socket after three consecutive request timeouts and drops the client, so the next lazy connect dials a fresh one. The proof is by deadlock analysis — in the absence of *any* wedge-detection transition, *probeli* finds half-open to be a reachable terminal state, and that terminal state *is* *biff-tww*. With either recovery transition present, half-open is deadlock-free.

2 Basic Types

$[USER, TTY, REPO, KVKEY, KVVALUE, SESSIONKEY]$

USER identifies a human user. *TTY* identifies a terminal session (random 8-char hex). *REPO* scopes all state to a repository. *KVKEY* and *KVVALUE* are opaque KV store keys and values. *SESSIONKEY* is the composite $\{user\} : \{tty\}$ identifier.

3 Free Types

$ZBOOL ::= ztrue \mid zfalse$

$ConnState ::= disconnected \mid reconnecting \mid connected \mid halfOpen \mid closed$

The connection has five states. *disconnected* is a deliberate, reversible release of the TCP connection (the relay's own `disconnect()`); a subsequent call reconnects. *reconnecting* is distinct: *nats-py*'s background reconnect loop is in progress after a dropped socket — `is_closed` is false, `is_connected` is false, and cached handles have already been invalidated by `_on_disconnect`. *connected* is an active TCP connection whose JetStream and KV handles are provisioned. *halfOpen* is the wedge: the socket is up (`is_closed` false) but every JetStream or KV request times out; the cached handles are stale and must not be used. *closed* is permanent shutdown — no reconnection is possible.

$ReqOutcome ::= reqOk \mid reqTimeout$

The outcome of a JetStream or KV request issued on a *connected* connection. *reqOk* is a normal response. *reqTimeout* is a `nats.errors.TimeoutError`: the request was sent but no reply arrived within the deadline, which is the observable signature of a half-open connection.

$WatcherPhase ::= idle \mid snapshotting \mid streaming \mid stopped$

The KV watcher progresses through phases: *idle* (not yet started), *snapshotting* (receiving the initial KV snapshot), *streaming* (receiving live updates post-snapshot), and *stopped* (shutdown complete).

$$KVOp ::= kvPut \mid kvDelete \mid kvPurge$$

Operations observed on KV entries. *kvPut* stores or updates a value. *kvDelete* and *kvPurge* remove entries (TTL expiry produces *kvDelete*).

$$WatcherEvent ::= weEntry \mid weSnapshotDone \mid weTimeout \mid weError \mid weShutdown$$

Events that the KV watcher loop must handle. *weEntry* is a KV change. *weSnapshotDone* is the *None* marker from `watcher.updates()`. *weTimeout* is a `TimeoutError` from idle periods. *weError* is an unrecoverable NATS error. *weShutdown* is the shutdown signal.

4 Global Constants

$kvTtlSeconds : \mathbb{N}$ $watcherTimeoutSeconds : \mathbb{N}$ $maxSessions : \mathbb{N}$ $restartDelaySeconds : \mathbb{N}$
$kvTtlSeconds = 259200$ $watcherTimeoutSeconds = 5$ $maxSessions = 1000$ $restartDelaySeconds = 2$

kvTtlSeconds is the NATS KV TTL (3 days). *watcherTimeoutSeconds* is the `updates()` timeout that keeps the shutdown check responsive. *maxSessions* bounds the model for animation.

5 Connection State

$Connection$ $connState : ConnState$ $kvProvisioned : ZBOOL$ $streamsProvisioned : ZBOOL$ $namesProvisioned : ZBOOL$ $wtmpAvailable : ZBOOL$
$connState \neq connected \Rightarrow kvProvisioned = zfalse \wedge streamsProvisioned = zfalse \wedge namesProvisioned = zfalse$ $connState = connected \Rightarrow kvProvisioned = ztrue \wedge streamsProvisioned = ztrue \wedge namesProvisioned = ztrue$

This is the generalised handle-validity invariant: valid JetStream, sessions-KV, and names-KV handles exist *if and only if* the connection is *connected*. In particular they do not exist in *halfOpen* — the stale handles cached during the wedge are, in the model, no handles at all. This generalises the earlier “*disconnected* or *closed* $\Rightarrow$ no handles” (DES-029) to cover *reconnecting* and *halfOpen* as well. It is the formal statement that a half-open connection must never be treated as usable: the biff-tww crash-loop arose precisely because the code kept and re-used handles that this invariant forbids. *wtmpAvailable* degrades independently — *wtmp* provisioning failure is non-fatal — and is cleared whenever the handles are.

$InitConnection$ $Connection'$ $connState' = disconnected$ $kvProvisioned' = zfalse$ $streamsProvisioned' = zfalse$ $namesProvisioned' = zfalse$ $wtmpAvailable' = zfalse$
--

6 Connection Operations

6.1 Ensure Connected (Lazy Connect)

<i>EnsureConnected</i>
$\Delta\textit{Connection}$
$\textit{connState} = \textit{disconnected} \vee \textit{connState} = \textit{connected}$
$\textit{connState}' = \textit{connected}$
$\textit{kvProvisioned}' = \textit{ztrue}$
$\textit{streamsProvisioned}' = \textit{ztrue}$
$\textit{namesProvisioned}' = \textit{ztrue}$
$\textit{wtmpAvailable}' = \textit{wtmpAvailable}$

The relay's lazy connect (`_ensure_connected`). It is enabled only from *disconnected* (a fresh connect) or *connected* (idempotent no-op). It is *not* enabled from *halfOpen*: this is the formal counterpart of the DES-029 bug. The fast path sees `is_closed` false and returns the stale cached handles without reconnecting, so *EnsureConnected* makes no progress out of the wedge and must not be modelled as if it did. Nor is it enabled from *reconnecting* (nats-py owns that window; provisioning would block until reconnect completes, bounded by `_CONNECT_PROVISION_TIMEOUT`, biff-wr3) or from *closed* (no reconnection after close). *wtmpAvailable* is preserved — it only changes during provisioning.

6.2 Disconnect (Reversible)

<i>Disconnect</i>
$\Delta\textit{Connection}$
$\textit{connState} = \textit{connected}$
$\textit{connState}' = \textit{disconnected}$
$\textit{kvProvisioned}' = \textit{zfalse}$
$\textit{streamsProvisioned}' = \textit{zfalse}$
$\textit{namesProvisioned}' = \textit{zfalse}$
$\textit{wtmpAvailable}' = \textit{zfalse}$

The relay's explicit, reversible `disconnect()`: it acts on a healthy *connected* connection and releases it. The next relay call reconnects via *EnsureConnected*. It is deliberately *not* the escape from *halfOpen* — the running server does not autonomously call `disconnect()` while wedged. The wedge is left only by autonomous detection: the keepalive path (*WedgeDetected*) or the proactive force-close (*ForceReconnect*), never by an ordinary `disconnect()`. Note, however, that *disconnected* is the state *ForceReconnect* moves *to*: a force-closed socket with its client dropped is, like a deliberate `disconnect()`, a connection whose next call must dial afresh.

6.3 Close (Permanent)

<i>Close</i>
$\Delta\textit{Connection}$
$\textit{connState} \neq \textit{halfOpen}$
$\textit{connState}' = \textit{closed}$
$\textit{kvProvisioned}' = \textit{zfalse}$
$\textit{streamsProvisioned}' = \textit{zfalse}$
$\textit{namesProvisioned}' = \textit{zfalse}$
$\textit{wtmpAvailable}' = \textit{zfalse}$

Permanent shutdown, reachable from any *definite* transport state. The precondition $\textit{connState} \neq \textit{halfOpen}$ records a modelling decision that is load-bearing for the liveness proof: an orderly close cannot be the escape from the wedge. In the biff-tww crash-loop the server never reaches an orderly `close()` — it is blocked awaiting a request that will never return — so *halfOpen* must be escaped by autonomous wedge detection, not by *Close*. From *closed* the operation is an idempotent self-loop, which keeps *closed* deadlock-free.

6.4 Reset Infrastructure

<i>ResetInfrastructure</i>
Δ Connection
$connState \neq halfOpen$ $connState' = connState$ $kvProvisioned' = zfalse$ $streamsProvisioned' = zfalse$ $namesProvisioned' = zfalse$ $wtmpAvailable' = zfalse$

Clears cached handles without changing the transport state (`reset_infrastructure()`). By the handle-validity invariant, $connState' = connState$ together with cleared handles forces $connState \neq connected$; the operation is therefore a self-loop at *disconnected*, *reconnecting*, and *closed*, where the handles are already absent. It is excluded from *halfOpen* for the same reason as *Close*: clearing a cached handle does not lift the wedge (the socket is still up and still unresponsive), so it must not be modelled as an escape.

6.5 Request (JetStream / KV Round-Trip)

<i>Request</i>
Δ Connection
$result! : ReqOutcome$
$connState = connected$ $(result! = reqOk \wedge connState' = connected \wedge kvProvisioned' = ztrue \wedge streamsProvisioned' = ztrue \wedge namesProvisioned' = ztrue) \vee$ $(result! = reqTimeout \wedge connState' = halfOpen \wedge kvProvisioned' = zfalse \wedge streamsProvisioned' = zfalse \wedge namesProvisioned' = zfalse)$

Every JetStream or KV round-trip issued by the relay — `stream_info` in `get_unread_summary`, `kv.get` in `heartbeat`, and the like. From *connected* it has two outcomes. *reqOk* leaves the connection *connected* with its handles intact. *reqTimeout* is the half-open signature: the request timed out, the connection moves to *halfOpen*, and the handles are invalidated to satisfy the handle-validity invariant. This is the operation that makes *halfOpen* reachable, and the crash-loop of *biff-tww* is exactly *reqTimeout* fired repeatedly with no way out.

6.6 Wedge Detected (Recovery)

<i>WedgeDetected</i>
Δ Connection
$connState = halfOpen$ $connState' = reconnecting$ $kvProvisioned' = zfalse$ $streamsProvisioned' = zfalse$ $namesProvisioned' = zfalse$ $wtmpAvailable' = zfalse$

The keepalive-driven recovery transition. Keepalive PINGs (`ping_interval=20, max_outstanding_pings=3`) detect that the server has stopped answering within ~ 60 s and fire `nats-py`'s `_on_disconnect`, which invalidates the cached handles and hands the socket to `nats-py`'s infinite reconnect loop. Before the proactive detector this was the *only* operation enabled in *halfOpen*; removing it (the pre-fix code, whose `ping_interval` default of 240s never fired in practice) made *halfOpen* a terminal state, and `probeli` surfaced that terminal state — *biff-tww* — as a reachable deadlock (the frozen `nats-relay-passA-nowedge` artifacts). *ForceReconnect* (below) now adds a second, faster escape from the same state; with either operation present, *halfOpen* is deadlock-free.

6.7 Reconnect (nats-py Auto-Reconnect Completes)

<i>Reconnect</i>
$\Delta Connection$
$connState = reconnecting$ $connState' = connected$ $kvProvisioned' = ztrue$ $streamsProvisioned' = ztrue$ $namesProvisioned' = ztrue$ $wtmpAvailable' = wtmpAvailable$

nats-py's background reconnect succeeds and fires `_on_reconnect`; the next relay call re-provisions the handles. This closes the recovery path $connected \rightarrow halfOpen \rightarrow reconnecting \rightarrow connected$, so no reachable state is stuck in *halfOpen*.

6.8 Force Reconnect (Proactive Wedge Teardown)

<i>ForceReconnect</i>
$\Delta Connection$
$connState = halfOpen$ $connState' = disconnected$ $kvProvisioned' = zfalse$ $streamsProvisioned' = zfalse$ $namesProvisioned' = zfalse$ $wtmpAvailable' = zfalse$

The proactive escape from the wedge (`_force_reconnect`, biff-3hp). Where *WedgeDetected* waits for nats-py's keepalive to declare the socket dead — a ~ 60 – 80 s floor — this operation acts on the relay's own evidence: after `_WEDGE_FORCE_RECONNECT_THRESHOLD = 3` consecutive runtime request timeouts on a still-connected socket (`is_connected` still true), `_tracked` calls `safe_close(nc)` and drops the client (`_nc`, `_js`, `_kv`, `_names_kv` all set to *None*). Each timed-out request blocks for the JetStream request timeout (~ 5 s), so the bound is $3 \times 5 \text{ s} \approx 15 \text{ s}$ — four to five times faster than the keepalive path.

The target state is *disconnected*, not *reconnecting*. This is the load-bearing distinction between the two escapes. *WedgeDetected* leaves the *same* client in nats-py's hands: the socket is handed to the background reconnect loop, so the model passes through *reconnecting* and recovers by *Reconnect*. *ForceReconnect* discards the client entirely; there is no socket left for nats-py to reconnect. Recovery is therefore *not* a new transition but the existing lazy connect: from *disconnected*, the next relay call fires *EnsureConnected*, whose slow path (`_cached_handles` returns *None* because `_nc` is *None*) dials a fresh `nats.connect` and re-provisions. The full proactive path is thus $connected \rightarrow halfOpen \rightarrow disconnected \rightarrow connected$, closed by operations the model already contains.

The three-consecutive-timeout threshold is not modelled as a counter, and deliberately so. The keepalive detector has its own three-missed-PING criterion (`max_outstanding_pings`), which the model abstracts into the single enabling of *WedgeDetected* from *halfOpen* rather than counting PINGs in the state. Modelling one detector's threshold as explicit state while abstracting the other's would be asymmetric, and it would inflate the state space without strengthening the liveness property, which asks only that *some* operation be enabled in *halfOpen*. The threshold is a latency parameter of the detector, recorded here in prose exactly as `ping_interval` is recorded for *WedgeDetected* — not a component of *Connection*. The handle-validity invariant is preserved: *ForceReconnect* lands in *disconnected* with every handle cleared, so no valid handle survives the wedge.

7 Session Presence

SessionStore

$sessions : SESSIONKEY \mapsto KVVALUE$
 $wallValue : KVKEY \mapsto KVVALUE$

$\#sessions \leq maxSessions$
 $\#wallValue \leq 1$

Sessions are a partial injection: each session key maps to exactly one value (the serialised *UserSession*). *wallValue* is either empty (no wall) or a single mapping (active wall). Initially the store is empty: $sessions = \emptyset$ and $wallValue = \emptyset$. There is no separate *InitSessionStore* schema: the model-checked state is the *Connection* machine (see §10), and the session store is exercised by the operations below only while $connState = connected$.

7.1 Update Session (Heartbeat / Login)

UpdateSession

$\Delta SessionStore$
 $\exists Connection$
 $key? : SESSIONKEY$
 $value? : KVVALUE$

$connState = connected$
 $sessions' = sessions \oplus \{key? \mapsto value?\}$
 $wallValue' = wallValue$
 $\#sessions' \leq maxSessions$

Creates or refreshes a session. Override ($\oplus$) ensures idempotent updates.

7.2 Delete Session (Logout / TTL Expiry)

DeleteSession

$\Delta SessionStore$
 $\exists Connection$
 $key? : SESSIONKEY$

$connState = connected$
 $key? \in \text{dom } sessions$
 $sessions' = \{key?\} \triangleleft sessions$
 $wallValue' = wallValue$

Removes a session. In production, this is triggered by explicit delete or KV TTL expiry.

7.3 Set Wall

SetWall

$\Delta SessionStore$
 $\exists Connection$
 $wallKey? : KVKEY$
 $wallVal? : KVVALUE$

$connState = connected$
 $sessions' = sessions$
 $wallValue' = \{wallKey? \mapsto wallVal?\}$

7.4 Clear Wall

ClearWall

Δ *SessionStore*

Ξ *Connection*

connState = *connected*

sessions' = *sessions*

wallValue' = $\emptyset$

8 KV Watcher State Machine

This section models the watcher loop from `_run_kv_watch` and its supervisor `_kv_watcher_loop`.

Watcher

phase : *WatcherPhase*

shutdownRequested : *ZBOOL*

restartCount : $\mathbb{N}$

restartCount ≤ 100

phase = *stopped* $\Rightarrow$ *shutdownRequested* = *ztrue*

The watcher tracks its current phase, whether shutdown has been requested, and how many times the supervisor has restarted it. A stopped watcher implies shutdown was requested. Initially the watcher is *idle*, with *shutdownRequested* = *zfalse* and *restartCount* = 0. As with the session store, there is no separate *InitWatcher* schema; the watcher operations are enabled only while *connState* = *connected*, which is consistent with the composed invariant *phase* $\neq$ *idle* $\Rightarrow$ *connState* = *connected*.

8.1 Start Watching

StartWatching

Δ *Watcher*

Ξ *Connection*

connState = *connected*

phase = *idle*

shutdownRequested = *zfalse*

phase' = *snapshotting*

shutdownRequested' = *zfalse*

restartCount' = *restartCount*

Transitions from *idle* to *snapshotting* when the watcher is created. The first entries from `watchall()` are the snapshot.

8.2 Snapshot Done

SnapshotDone

Δ *Watcher*

Ξ *Connection*

connState = *connected*

phase = *snapshotting*

shutdownRequested = *zfalse*

phase' = *streaming*

shutdownRequested' = *zfalse*

restartCount' = *restartCount*

The *None* marker from `updates()` signals the end of the snapshot. **The watcher continues into streaming phase.** This is the key property: the old `async for` pattern terminated here (biff-udp).

8.3 Receive Entry

<i>ReceiveEntry</i>
Δ <i>Watcher</i>
Ξ <i>Connection</i>
<i>connState</i> = <i>connected</i> (<i>phase</i> = <i>snapshotting</i> $\vee$ <i>phase</i> = <i>streaming</i>) <i>shutdownRequested</i> = <i>zfalse</i> <i>phase'</i> = <i>phase</i> <i>shutdownRequested'</i> = <i>zfalse</i> <i>restartCount'</i> = <i>restartCount</i>

A KV entry is received. The phase does not change — entries arrive during both snapshotting and streaming.

8.4 Handle Timeout

<i>HandleTimeout</i>
Δ <i>Watcher</i>
Ξ <i>Connection</i>
<i>connState</i> = <i>connected</i> <i>phase</i> = <i>streaming</i> <i>shutdownRequested</i> = <i>zfalse</i> <i>phase'</i> = <i>streaming</i> <i>shutdownRequested'</i> = <i>zfalse</i> <i>restartCount'</i> = <i>restartCount</i>

A `TimeoutError` from `updates(timeout=5.0)` is caught and the loop continues. **The watcher stays in streaming phase.** Without the catch, this would propagate to the supervisor and trigger a restart.

8.5 Handle Error (Supervisor Restart)

<i>HandleError</i>
Δ <i>Watcher</i>
Ξ <i>Connection</i>
<i>connState</i> = <i>connected</i> <i>phase</i> $\neq$ <i>idle</i> <i>phase</i> $\neq$ <i>stopped</i> <i>shutdownRequested</i> = <i>zfalse</i> <i>phase'</i> = <i>snapshotting</i> <i>shutdownRequested'</i> = <i>zfalse</i> <i>restartCount'</i> = <i>restartCount</i> + 1

An unrecoverable error causes the supervisor to restart the watcher. The new watcher begins in snapshotting phase (a fresh `watchall()`).

8.6 Request Shutdown

<i>RequestShutdown</i>
Δ <i>Watcher</i>
Ξ <i>Connection</i>
<i>connState</i> = <i>connected</i>
<i>shutdownRequested</i> = <i>zfalse</i>
<i>shutdownRequested'</i> = <i>ztrue</i>
<i>phase'</i> = <i>stopped</i>
<i>restartCount'</i> = <i>restartCount</i>

The shutdown event signals the watcher to exit. The loop checks `shutdown.is_set()` on every iteration.

9 Composed System State

<i>NatsRelayState</i>
<i>Connection</i>
<i>SessionStore</i>
<i>Watcher</i>
<i>phase</i> $\neq$ <i>idle</i> $\Rightarrow$ <i>connState</i> = <i>connected</i>
<i>phase</i> = <i>streaming</i> $\Rightarrow$ <i>kvProvisioned</i> = <i>ztrue</i>

The watcher can only be active when the connection is established. Streaming implies KV is provisioned (the watcher reads from it). With the generalised handle-validity invariant, *phase* = *streaming* $\Rightarrow$ *kvProvisioned* = *ztrue* also implies *phase* = *streaming* $\Rightarrow$ *connState* = *connected*: a wedge (*halfOpen*, where *kvProvisioned* = *zfalse*) cannot coexist with a streaming watcher, which is consistent — a wedge manifests to the watcher as a *weError* and drives it through *HandleError*.

The composed initial state is: *connState* = *disconnected* with no handles (*InitConnection*), an empty session store, and an *idle* watcher. *InitConnection* is the sole initialisation schema so that the model checker explores the *Connection* lifecycle as a single, well-posed state machine; the session and watcher components are exercised as connection-gated operations rather than as independent initial states (see §10).

10 Key Properties and Model Checking

The specification captures these properties, each of which can be verified by model checking with `probcli (z-spec test)`:

1. **Handle validity:** valid JetStream, sessions-KV, and names-KV handles exist if and only if *connState* = *connected*. A *halfOpen*, *disconnected*, *reconnecting*, or *closed* connection has no valid handles. This generalises DES-029 and is the safety half of the biff-tww fix: stale handles cached during a wedge are, in the model, no handles at all.
2. **Wedge liveness (biff-tww):** no reachable state is stuck in *halfOpen*. Two independent recovery paths leave the wedge: the keepalive path *WedgeDetected* $\rightarrow$ *Reconnect* (via *reconnecting*, on the same client) and the proactive path *ForceReconnect* $\rightarrow$ *EnsureConnected* (via *disconnected*, on a fresh client). Each is always enabled in *halfOpen*, so the connection cannot deadlock in the wedge. The proof is by deadlock analysis: removing *every* operation enabled in *halfOpen* (the pre-fix code, whose keepalive ping never fired in time, with no proactive detector) makes *halfOpen* a reachable terminal state. `probcli` surfaces it as a deadlock along the trace *EnsureConnected* $\rightarrow$ *Request(reqTimeout)*, which is biff-tww. With either recovery transition present the state space is deadlock-free.
3. **Snapshot survival:** *SnapshotDone* transitions to *streaming*, never to *idle* or *stopped*. The watcher remains alive.
4. **Timeout resilience:** *HandleTimeout* keeps the watcher in *streaming* phase. No restart, no phase change.

5. **Error restart:** Only *HandleError* increments *restartCount*. Snapshot-done and timeout do not.
6. **Connection prerequisite:** The watcher cannot be active (*phase* $\neq$ *idle*) when the connection is not *connected*; every session and watcher operation is enabled only while *connState* = *connected*.
7. **Session uniqueness:** The partial injection ($\succ\leftrightarrow$) on sessions guarantees each session key maps to exactly one value.